INTEGRATING BLOCKCHAIN AND SOFTWARE DEFINED NETWORK TUTORIAL

I. Introduction

II. Prerequisites

- Basic understanding of Software Defined Network
- Familiarity with blockchain concepts
- A funded Tezos wallet on Ghostnet
- Installed tools:
  - Python3.8
  - Git

1. Downloading and installing Mininet-WiFi:

Mininet is a network emulator that supports the creation of wireless networks. Follow these steps to install it:

```
~$ git clone https://github.com/mininet/mininet.git
~$ cd mininet
~/mininet$ sudo util/install.sh -fnv
// Finally, verify that Mininet is installed and working in the VM:
~/mininet$ sudo mn --test pingall
```

Mininet relies on Linux Kernel components to function properly. Of the different Linux distributions that can be used to this end, we recommend Ubuntu, since Mininet was extensively tested on it.

2. Downloading and installing Pytezos

Pytezos is a Python library for interacting with Tezos blockchain, testing smart contracts, and writing Michelson scripts.

To install, run following command on Linux distributions:

```
~$ pip install pytezos
```

III. Implementation

1. File structure

Here is the link to main folder of the project: https://gitlab.com/herohthd/mininet-tezos

```
sdn/
├── sdn.py
├── network_information.json
├── simulation_complete.signal
├── contract/
│   ├── contract.py
├── pytezos/
│   ├── server.py
```

```
└── .gitignore
```

2. Set Up the SDN

We will guide you through setting up a Software-Defined Network (SDN) with Mininet. We will create a custom topology with multiple regions, where each region has several nodes and switches, establish links between regions and hosts within each region, and export the network information to a JSON file.

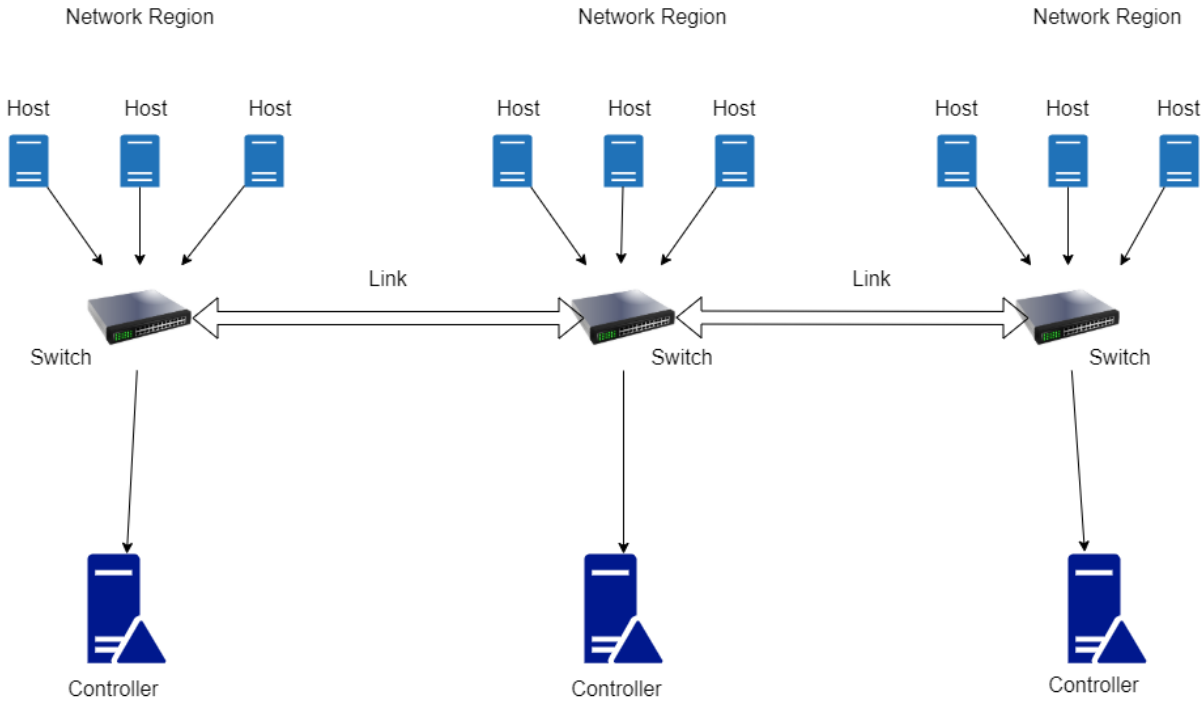


Figure 1: SDN Structure

Import Necessary Modules

The script starts by importing necessary modules:

```
from mininet.topo import Topo
from mininet.net import Mininet
from mininet.node import Controller, RemoteController, OVSKernelSwitch
from mininet.link import TCLink
from mininet.cli import CLI
from mininet.log import setLogLevel
import threading
import json
import time
```

Define Constants and Helper Functions

Set the number of regions and define helper functions to determine the region based on the node name:

```
NUMBER_OF_REGIONS = 3
def get_region_from_switch(node_name):
    if node_name.startswith('s') and node_name[1].isdigit():
        return f"region_{node_name[1]}"
```

```
    return None

def get_region_from_host(node_name):
    if node_name.startswith('h') and node_name[1].isdigit():
        return f"region_{node_name[1]}"
    return None
```

Define the Custom Topology

Define a custom topology class that extends Topo and builds the network with regions and links:

```
class CustomTopo(Topo):
    def build(self):
        # Create three network sub-regions, each with 3 nodes
        regions = {}
        switches = []
        for i in range(1, NUMBER_OF_REGIONS + 1):
            region = [self.addHost(f'h{i}{j}') for j in range(1, 4)]
            regions[f'region_{i}'] = region
            switch = self.addSwitch(f's{i}')
            switches.append(switch)
            for host in region:
                self.addLink(host, switch, bw=10, delay='5ms')

        # Add links between regions
        self.addLink(switches[0], switches[1], bw=20, delay='10ms')
        self.addLink(switches[1], switches[2], bw=30, delay='15ms')
        self.addLink(switches[2], switches[0], bw=40, delay='20ms')
```

Export Network Information

Define a function to export network information to a JSON file:

```
def export_network_info(net, topo, filename='network_info.json'):
    signal_file_path = "simulation_complete.signal"

    regions = {}
    for i in range(1, NUMBER_OF_REGIONS + 1):
        region_name = f'region_{i}'
        regions[region_name] = {'nodes': {}, 'links': {}}

    # Assign nodes to regions based on your topology
    for host in net.hosts:
        region = get_region_from_host(host.name)
        regions[region]['nodes'][host.name] = {
            'IP': host.IP(),
            'MAC': host.MAC(),
        }

    # Assign links to regions based on stored link info
```

```
    for link in net.links[:-NUMBER_OF_REGIONS]:
        intf1, intf2 = link.intf1, link.intf2
        # Print additional link parameters if available
        if isinstance(link, TCLink):
            link_info = link.intf1.params  # Access link parameters
            region1 = get_region_from_switch(intf1.node.name)
            region2 = get_region_from_switch(intf2.node.name)
            region = region1 or region2
            if region:
                regions[region]['links'][f"{intf1.node.name}-{intf2.node.name}"] = {
                    'bw': link_info.get('bw', 'unknown'),
                    'delay': link_info.get('delay', 'unknown')
                }

    for link in net.links[-NUMBER_OF_REGIONS:]:
        intf1, intf2 = link.intf1, link.intf2
        if isinstance(link, TCLink):
            link_info = link.intf1.params  # Access link parameters
            region1 = get_region_from_switch(intf1.node.name)
            region2 = get_region_from_switch(intf2.node.name)
            regions[region1]['links'][f"{intf1.node.name}-{intf2.node.name}"] = {
                'bw': link_info.get('bw', 'unknown'),
                'delay': link_info.get('delay', 'unknown')
            }
            regions[region2]['links'][f"{intf2.node.name}-{intf1.node.name}"] = {
                'bw': link_info.get('bw', 'unknown'),
                'delay': link_info.get('delay', 'unknown')
            }

    # Save to a JSON file
    with open(filename, 'w') as f:
        json.dump(regions, f, indent=4)

    with open(signal_file_path, "w"):
        pass

    time.sleep(600)
```

Run the Network

Define the main function to set up and run the network:

```
def run():
    topo = CustomTopo()
    net = Mininet(topo=topo, switch=OVSKernelSwitch, link=TCLink, build=False)

    # Create controllers for each region
    controllers = []
    for i in range(1, NUMBER_OF_REGIONS + 1):
```

```
        controller = net.addController(f'c{i}', controller=Controller, ip='127.0.0.1', port=6633 +
i)
        controllers.append(controller)

    net.build()

    # Assign controllers to switches
    for i, switch in enumerate(net.switches):
        switch.start([controllers[i]])

    net.start()

    # Export network information
    file_path = "network_information.json"
    tracking_thread = threading.Thread(target=export_network_info, args=(net, topo))
    tracking_thread.start()

    # Start CLI
    CLI(net)
    net.stop()

if __name__ == '__main__':
    setLogLevel('info')
    run()
```

3. Set Up the Smart Contract

In this part, we will walk through a SmartPy smart contract designed to store and manage network information. We will explain the structure and functionality of the contract and guide you on how to deploy and interact with it.

a) Overview of the Smart Contract

The provided smart contract is written in SmartPy, a Python-like language used to write smart contracts for the Tezos blockchain. This contract has one main functionality:

1. Storing network information of each network region.

To implement and test the contract, we recommend using Smartpy IDE, an intuitive and powerful smart contract development IDE for Tezos blockchain.

b) The Smart Contract Code

Here is the full code of the smart contract:

```
import smartpy as sp

@sp.module
def main():
```

```
    class NetworkInfoContract(sp.Contract):
        def __init__(self):
            self.data.regions = {}


        @sp.entrypoint
        def store_network_info(self, params):
            sp.cast(params, sp.record(region=sp.string,
                                      nodes=sp.map[sp.string, sp.record(IP=sp.string, MAC=sp.string)],
                                      links=sp.map[sp.string, sp.record(bw=sp.int, latency=sp.string)]
                                     ))
            self.data.regions[params.region] = sp.record(nodes=params.nodes,
links=params.links)

if "templates" not in __name__:
    @sp.add_test()
    def test():
        sc = sp.test_scenario("Network Info Test", main)
        c = main.NetworkInfoContract()
        sc += c

        sc.h1("NetworkInfoContract")
        sc.h2("Store Network Info")

        region_1_nodes_info = {
            'h11': sp.record(IP='10.0.0.1', MAC='00:00:00:00:00:01'),
            'h12': sp.record(IP='10.0.0.2', MAC='00:00:00:00:00:02'),
            'h13': sp.record(IP='10.0.0.3', MAC='00:00:00:00:00:03')
        }

        region_1_links_info = {
            'link1': sp.record(bw=10, latency='5ms'),
            'link2': sp.record(bw=10, latency='5ms'),
            'link3': sp.record(bw=10, latency='5ms')
        }

        region_2_nodes_info = {
            'h21': sp.record(IP='10.0.0.4', MAC='00:00:00:00:00:04'),
            'h22': sp.record(IP='10.0.0.5', MAC='00:00:00:00:00:05'),
            'h23': sp.record(IP='10.0.0.6', MAC='00:00:00:00:00:06')
        }

        region_2_links_info = {
            'link1': sp.record(bw=10, latency='5ms'),
            'link2': sp.record(bw=10, latency='5ms'),
            'link3': sp.record(bw=10, latency='5ms')
        }
```

```
    c.store_network_info(
        region='region_1',
        nodes=region_1_nodes_info,
        links=region_1_links_info
    )

    c.store_network_info(
        region='region_2',
        nodes=region_2_nodes_info,
        links=region_2_links_info
    )
```

c) Explanation of the Contract

Importing SmartPy and Defining the Module

The contract begins by importing the SmartPy library and defining a module named `main`:

```
import smartpy as sp

@sp.module
def main():
```

Contract Initialization

The contract class `NetworkInfoContract` is initialized with three empty maps: `timestamps`, `latest_infor`, and `distances`:

```
  class NetworkInfoContract(sp.Contract):
    def __init__(self):
      self.data.regions = {}
```

- `regions`: Stores network information of each region.

Storing Network Information

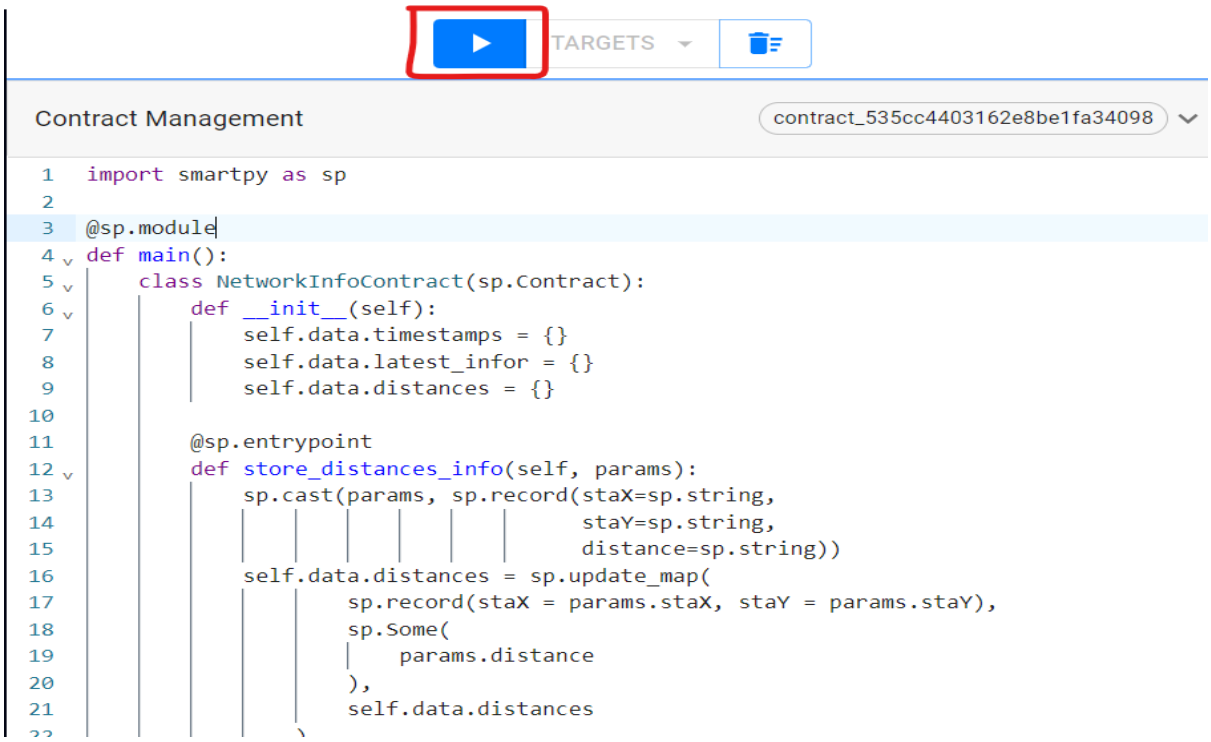
The `store_network_info` entrypoint function stores and updates network information for each region:

```
@sp.entrypoint
    def store_network_info(self, params):
      sp.cast(params, sp.record(region=sp.string,
                                nodes=sp.map[sp.string, sp.record(IP=sp.string, MAC=sp.string)],
                                links=sp.map[sp.string, sp.record(bw=sp.int, latency=sp.string)]
                               ))
      self.data.regions[params.region] = sp.record(nodes=params.nodes,
links=params.links)
```

- `params`: A record containing region name, nodes information including IP and MAC address, and link information with bandwidth and latency.

Testing

SmartPy provides a built-in testing framework that allows you to test your contracts before deploying them on the blockchain. To run the test, click on red circled button as shown on the below image.

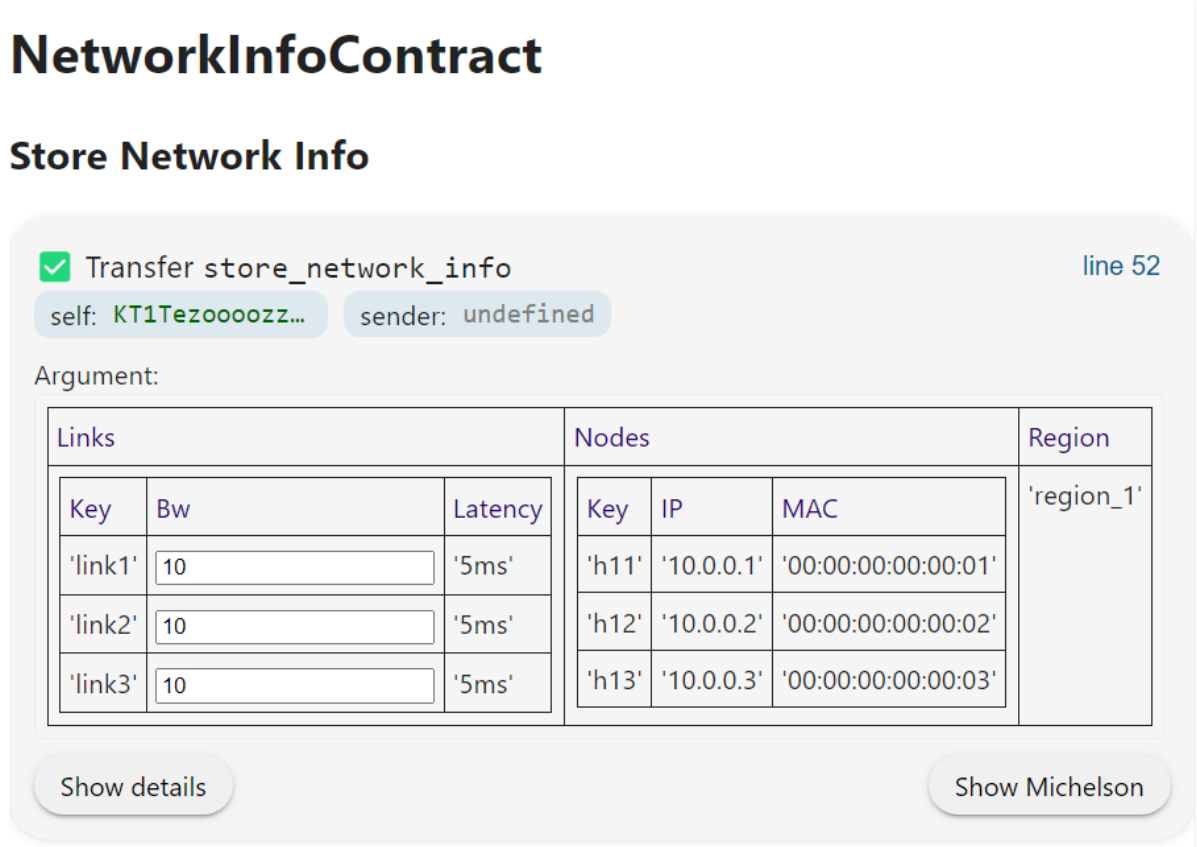



```
import smartpy as sp

@sp.module
def main():
    class NetworkInfoContract(sp.Contract):
        def __init__(self):
            self.data.timestamps = {}
            self.data.latest_infor = {}
            self.data.distances = {}

        @sp.entrypoint
        def store_distances_info(self, params):
            sp.cast(params, sp.record(staX=sp.string,
                                      staY=sp.string,
                                      distance=sp.string))
            self.data.distances = sp.update_map(
                sp.record(staX = params.staX, staY = params.staY),
                sp.Some(
                    params.distance
                ),
                self.data.distances
```

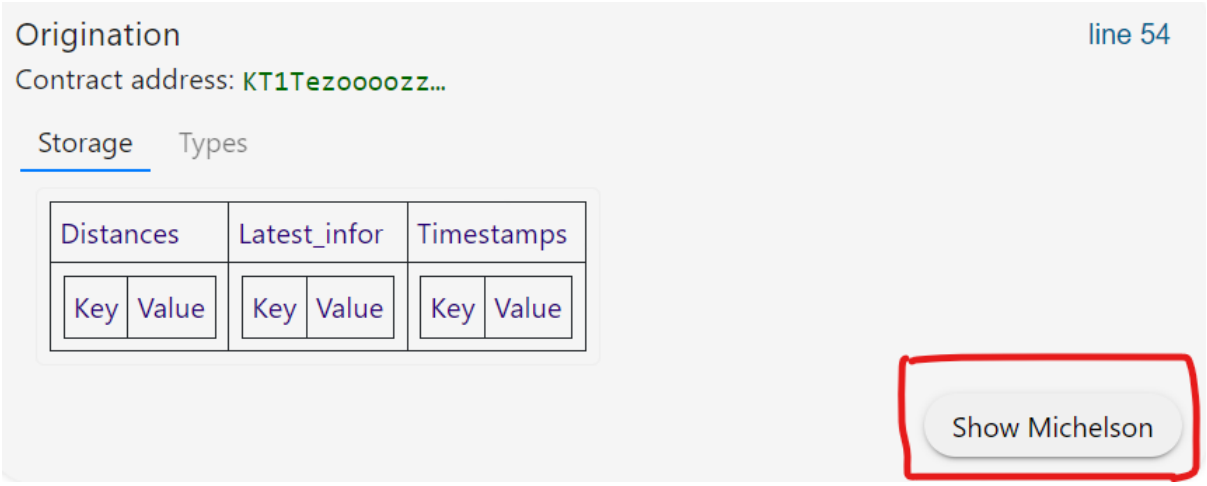
On the right side of the IDE, you can see the result of the tests.



As result, the test calls the `store_network_info` entrypoint multiple times to store network information for different regions.

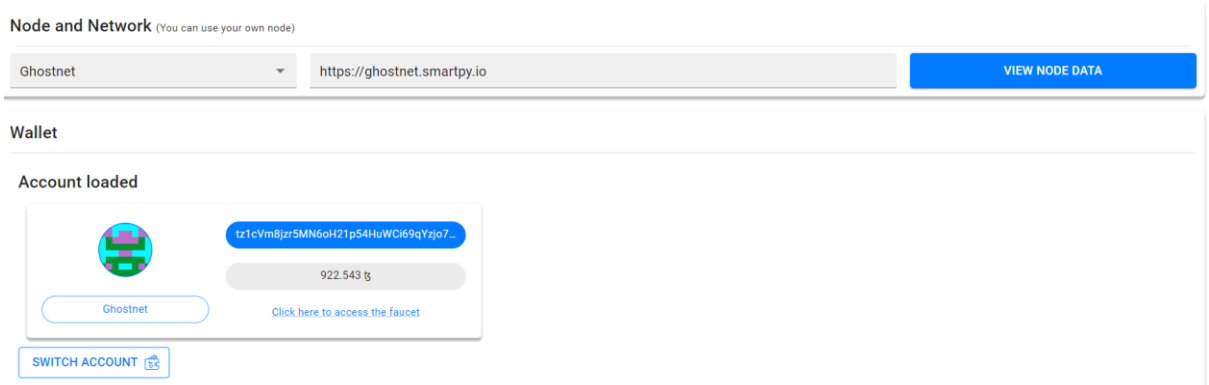
d) Deploy the contract on testnet

- Step 1: Click on **Show Michelson**

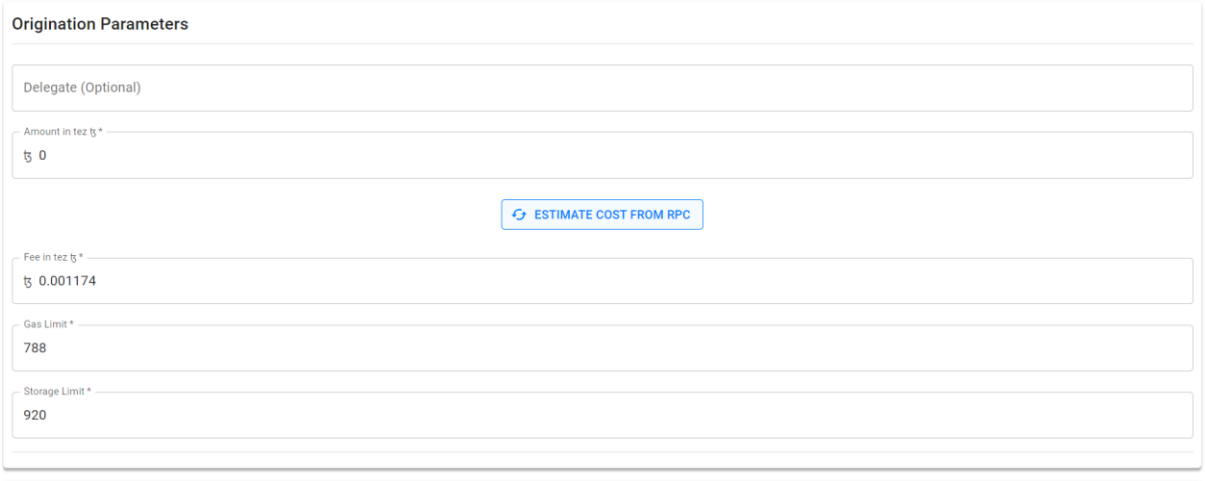


- Step 2: After that, it'll open a modal. Then click on **Deploy Contract.**

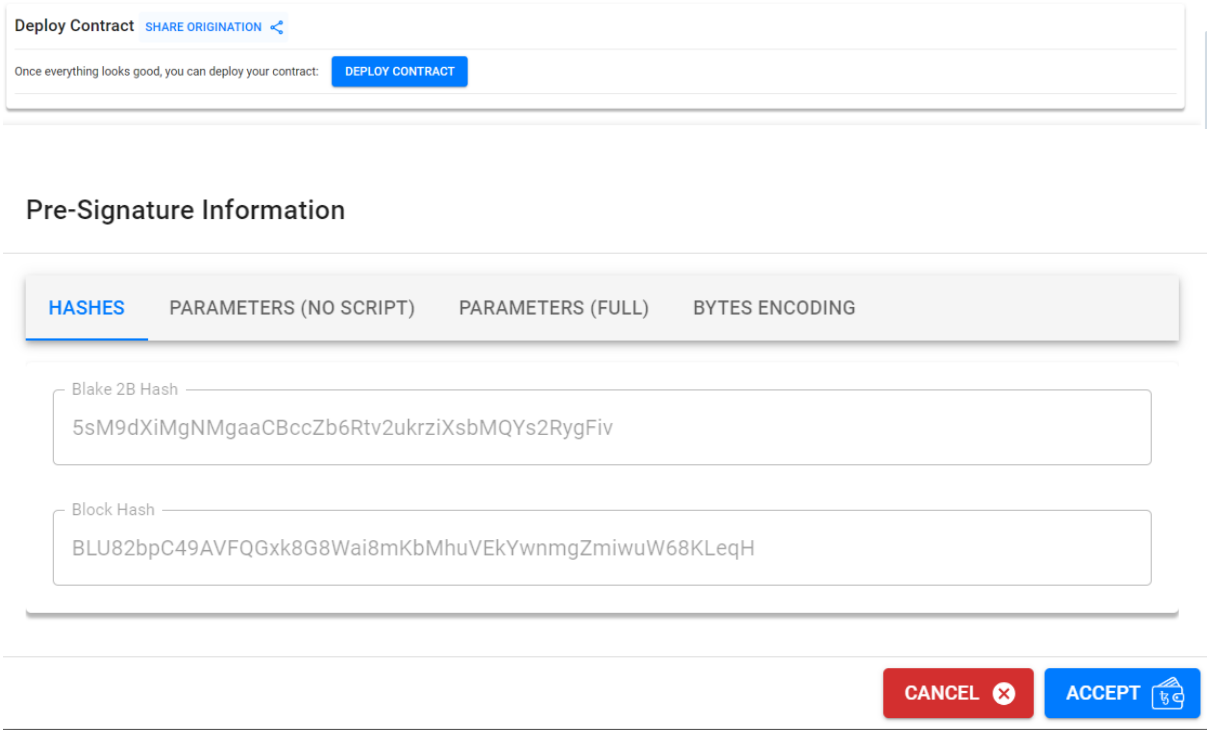
- Step 3: On deploying tab, select Ghostnet and connect your account.



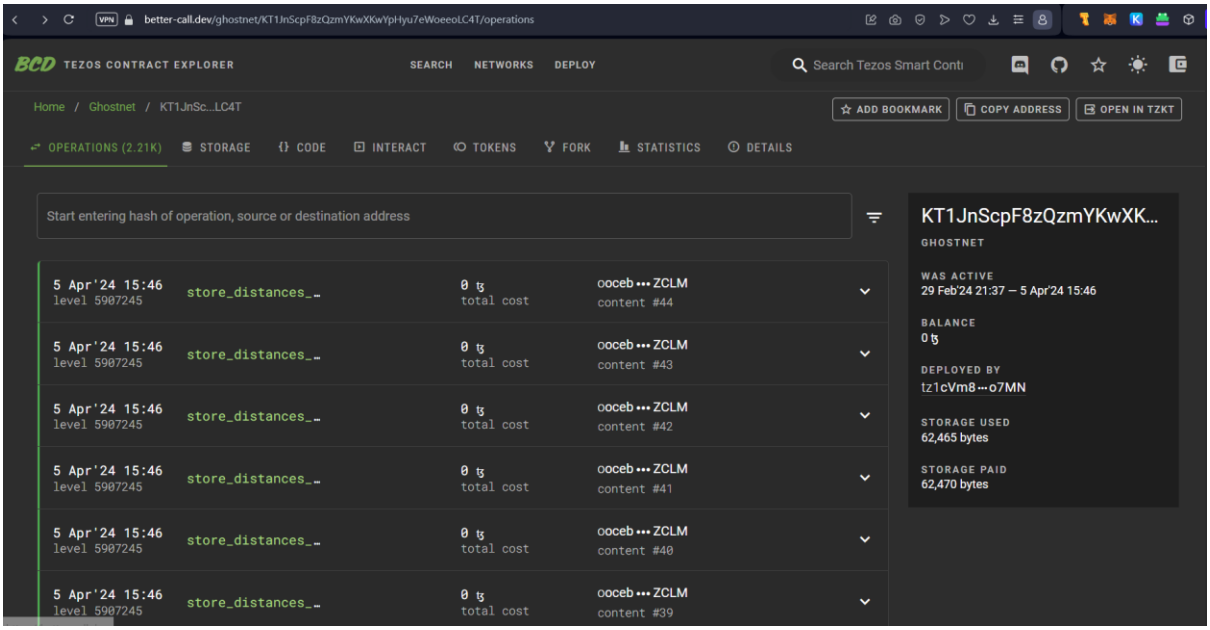
- Step 4: Then click **ESTIMATE COST FROM RPC** button



- Step 5: Sign transaction to deploy contract



After deploying, you should have a contract on blockchain which you can interact with via Better-call-dev, a Tezos smart contract explorer and search engine. Here's example of a deployed contract:



4. Integrate SDN with Smart contract

Store network information on the contract

This server script is designed to read network information from a JSON file and push it to a Tezos smart contract. The script runs continuously, listening for changes in the network information file and updating the smart contract whenever new information is available.

Key Components:

1. **Environment Setup:**
   - The script uses environment variables stored in a `.env` file for sensitive information like the Tezos key (PYTEZOS_KEY).
   - It also sets up the Tezos node URL and the smart contract address.
2. **Logging:**
   - Logging is configured to provide information about the server's operations and any errors that occur.
3. **Function Definitions:**
   - `read_network_information(file_path)`: Reads and returns the content of the JSON file specified by file_path.
   - `update_network_infor (network_info, contract)`: Updates the smart contract with the network information. It extracts timestamp, stations, and distances from the network info and sends this data to the smart contract using Tezos operations.
   - `listen_for_changes(file_path, contract)`: Continuously listens for changes in the network_information.json file. When a change is detected, it reads the file, updates the smart contract, and deletes the signal file to indicate that the update has been processed.
4. **Main Function:**
   - `main()`: Initializes the server, loads the smart contract, and starts a thread to listen for changes in the network information file.

How It Works:

1. **Environment Variables:**
   - The script loads the environment variables from a `.env` file using the python-dotenv library. This includes the Tezos private key required to sign transactions.
2. **Contract Initialization:**
   - The script connects to the Tezos network using the node URL and accesses the smart contract using the provided contract address.
3. **Reading Network Information:**
   - The `read_network_information` function reads the JSON file that contains the network information. This file is updated by the simulation periodically.
4. **Updating the Smart Contract:**
   - The `update_network_infor` function extracts relevant information from the network info and prepares Tezos operations to update the smart contract. It handles two types of information:
     - **Region Information:** Includes details like region name, node information and link information.
   - These operations are then sent to the Tezos blockchain in a bulk transaction to ensure atomicity and efficiency.
5. **Listening for Changes:**
   - The `listen_for_changes` function continuously checks for the presence of a signal file (simulation_complete.signal). This file indicates that new network information is available.
   - When the signal file is detected, it reads the network information from the JSON file, updates the smart contract, and deletes the signal file.
6. **Server Execution:**
   - The `main` function sets up the logging, initializes the connection to the smart contract, and starts a thread to listen for changes in the network information file.

- The main thread keeps running to ensure the server remains active.

```
import os
import time
import json
from dotenv import load_dotenv
from pytezos import pytezos
from threading import Thread
import logging
from datetime import datetime

# Load environment variables from .env
load_dotenv()

# Fetch the key from the environment variable
PYTEZOS_KEY = os.environ["PYTEZOS_KEY"]

if not PYTEZOS_KEY:
    raise ValueError("PYTEZOS_KEY environment variable is not set")

CONTRACT_ADDRESS = "KT1CHNKbwUxRowxvHkjtKwh1tgGxzQmxDZer"
CONTRACT_STORAGE = {"regions": {}}
NODE_URL = "https://ghostnet.ecadinfra.com"

# Configure logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def read_network_information(file_path):
    with open(file_path, "r") as json_file:
        return json.load(json_file)

def update_contract(network_info, contract):
    operations = []

    for region_name, region_info in network_info.items():
        # Nodes info
        nodes_info = {
            node_name: {
                'IP': node_info["IP"],
                'MAC': node_info["MAC"]
            }
            for node_name, node_info in region_info.get('nodes', {}).items()
        }

        # Links info
        links_info = {
            link_name: {
                "bw": link_info["bw"],
```

```
                "latency": link_info["delay"]
            }
            for link_name, link_info in region_info.get('links', {}).items()
        }

        # Add store_network_info operation
        operations.append(
            contract.store_network_info(
                region=region_name,
                nodes=nodes_info,
                links=links_info
            )
        )

    # Perform bulk operation for store_network_info calls
    pytezos.bulk(*operations).send(min_confirmations=1)
    logger.info(f"{datetime.now().strftime('%d-%m-%Y %H:%M:%S')} Bulk operation for
store_network_info completed")

def listen_for_changes(file_path, contract):
    signal_file_path = "simulation_complete.signal"

    while True:
        # Wait until the signal file is created by the simulator
        while not os.path.exists(signal_file_path):
            time.sleep(1)

        try:
            # Read the content of the JSON file
            network_info = read_network_information(file_path)

            # Update the contract with the new network information
            update_contract(network_info, contract)
            time.sleep(600)
        except Exception as e:
            logger.error(f"Error: {e}")

        # Remove the signal file to indicate the content has been read
        os.remove(signal_file_path)

def main():
    logger.info(f"{datetime.now().strftime('%d-%m-%Y %H:%M:%S')} Server is running")
    # Load the existing contract using the provided CONTRACT_ADDRESS
    py = pytezos.using(shell=NODE_URL)
    contract = py.contract(CONTRACT_ADDRESS)

    # Start a thread to listen for changes in the network_information.json file
    file_path = "/home/huydq/Mininet/mininet/mininet/sdn/network_info.json"
```

```
    update_thread = Thread(target=listen_for_changes, args=(file_path, contract))
    update_thread.start()

    # Keep the main thread alive
    while True:
        time.sleep(1)

if __name__ == "__main__":
    main()
```

IV. Running

To run the project, first on one terminal, we run the server:

```
~/mininet/sdn$ python3.8 pytezos/server.py
```

The server will start and listen to the updating network information signal:

After that, on other terminal, we run the SDN environment with updating network information thread.

```
~/mininet/sdn$ sudo python3 sdn.py
```

```
huydq@huydq-VirtualBox:~/Mininet/mininet/sdn$ sudo python3 sdn.py
*** Creating network
*** Adding hosts:
h11 h12 h13 h21 h22 h23 h31 h32 h33
*** Adding switches:
s1 s2 s3
*** Adding links:
(10.00Mbit 5ms delay) (10.00Mbit 5ms delay) (h11, s1) (10.00Mbit 5ms delay) (10.00
Mbit 5ms delay) (h12, s1) (10.00Mbit 5ms delay) (10.00Mbit 5ms delay) (h13, s1) (1
0.00Mbit 5ms delay) (10.00Mbit 5ms delay) (h21, s2) (10.00Mbit 5ms delay) (10.00Mb
it 5ms delay) (h22, s2) (10.00Mbit 5ms delay) (10.00Mbit 5ms delay) (h23, s2) (10.
00Mbit 5ms delay) (10.00Mbit 5ms delay) (h31, s3) (10.00Mbit 5ms delay) (10.00Mbit
 5ms delay) (h32, s3) (10.00Mbit 5ms delay) (10.00Mbit 5ms delay) (h33, s3) (20.00
Mbit 10ms delay) (20.00Mbit 10ms delay) (s1, s2) (30.00Mbit 15ms delay) (30.00Mbit
 15ms delay) (s2, s3) (40.00Mbit 20ms delay) (40.00Mbit 20ms delay) (s3, s1)
*** Configuring hosts
h11 h12 h13 h21 h22 h23 h31 h32 h33
*** Starting controller
c1 c2 c3
*** Starting 3 switches
s1 s2 s3 ...(10.00Mbit 5ms delay) (10.00Mbit 5ms delay) (10.00Mbit 5ms delay) (20.
00Mbit 10ms delay) (40.00Mbit 20ms delay) (10.00Mbit 5ms delay) (10.00Mbit 5ms del
ay) (10.00Mbit 5ms delay) (20.00Mbit 10ms delay) (30.00Mbit 15ms delay) (10.00Mbit
 5ms delay) (10.00Mbit 5ms delay) (10.00Mbit 5ms delay) (30.00Mbit 15ms delay) (40
.00Mbit 20ms delay)
*** Starting CLI:
```

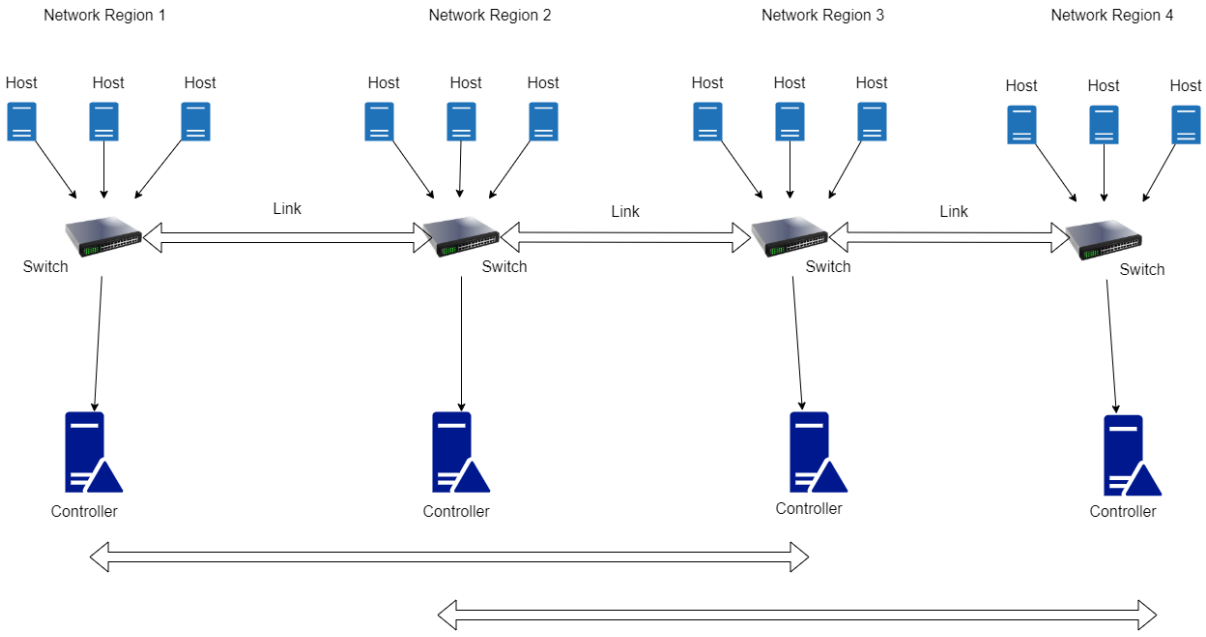
And finally, we have a running SDN environment with a server to push latest network information to the smart contract.
You can track the operation and current storage of the smart contract on better-call-dev.

V. Use case

In this tutorial, we'll walk through the steps to find the optimal region for indirect connections between hosts in different regions using a smart contract. We'll use a Mininet topology to simulate the network and a SmartPy contract to determine the best region to relay connections. This scenario involves four regions, each with hosts, a switch, and a controller.

Network Setup

1. **Network Topology:**
    - **Region 1:** Hosts (h11, h12, h13), Switch (s1), Controller (c1)
    - **Region 2:** Hosts (h21, h22, h23), Switch (s2), Controller (c2)
    - **Region 3:** Hosts (h31, h32, h33), Switch (s3), Controller (c3)
    - **Region 4:** Hosts (h41, h42, h43), Switch (s4), Controller (c4)
2. **Switch Connections:**
    - Switch s1 connects to s2 and s3.
    - Switch s4 connects to s2 and s3.
    - Switches s2 and s3 connect to all other regions.



As we can see, hosts from switch 1 cannot connect to ones from switch 4 due to their switch don’t link directly to each other. Therefore, hosts from switch 1 have to establish a connection to hosts from switch 2 or 3 to indirectly connect to hosts from switch 4. In this case, we use a smart contract to find the optimal region that switch 1 should establish the connection.

We'll modify the network topology and create a SmartPy contract to store network information and find the optimal region for indirect connections based on the least latency.

1. Implementation

a) sdnUseCase.py

In **sdnUseCase.py**, we have the same code as **sdn.py** and add controller object to regions:

```
# Store controller information
    for controller in net.controllers:
        region = f"region_{controller.name[-1]}"
        regions[region]["controller"][controller.name] = {
            'IP': controller.ip,
            'PORT': controller.port,
            'ADDRESS': CONTROLLER_ADDRESSES[int(controller.name[-1]) - 1]
        }
```

b) serverUseCase.py

In **serverUseCase.py**, we also have same code as **server.py** but add following code lines to store controller information to the smart contract

```
# Store controller information
    for controller in net.controllers:
        region = f"region_{controller.name[-1]}"
        regions[region]["controller"][controller.name] = {
            'IP': controller.ip,
            'PORT': controller.port,
            'ADDRESS': CONTROLLER_ADDRESSES[int(controller.name[-1]) - 1]
        }
```

c) contractUseCase.py

```
import smartpy as sp

@sp.module
def main():
    class NetworkInfoContract(sp.Contract):
        def __init__(self):
            self.data.regions = {}
            self.data.optimal_routes = {}
            self.data.region_switches = {
                "s1": "region_1",
                "s2": "region_2",
                "s3": "region_3",
                "s4": "region_4",
            }
            self.data.region_controllers = {
                "region_1": "c1",
                "region_2": "c2",
```

```
            "region_3": "c3",
            "region_4": "c4",
        }

    @sp.entrypoint
    def store_network_info(self, params):
        sp.cast(params, sp.record(region=sp.string,
                                  nodes=sp.map[sp.string, sp.record(IP=sp.string, MAC=sp.string)],
                                  links=sp.map[sp.string, sp.record(bw=sp.int, latency=sp.int)],
                                  controller=sp.map[sp.string, sp.record(IP=sp.string, PORT=sp.int,
ADDRESS= sp.address)],
                                ))
        self.data.regions[params.region] = sp.record(nodes=params.nodes,
links=params.links, controller=params.controller)

    @sp.entrypoint
    def find_optimal_region(self, params):
        sp.cast(params, sp.record(
            starter_region_name=sp.string,
            destination_region_name=sp.string,
        ))

        assert sp.amount == sp.mutez(10000), "Incorrect fee amount"
        assert self.data.regions.contains(params.starter_region_name), "No such starter
region"
        assert self.data.regions.contains(params.destination_region_name), "No such
destination region"

        # Extract the regions and their links
        regions = self.data.regions
        starter_region = regions[params.starter_region_name]
        destination_region = regions[params.destination_region_name]

        # Extract the regions and their links
        regions = self.data.regions
        starter_links = regions[params.starter_region_name].links
        destination_links = regions[params.destination_region_name].links

        optimal_region = ""
        min_latency = 99999
        starter_destination = params.starter_region_name + "_" +
params.destination_region_name

        # Check for links from starter region to other regions
        for link_name in starter_links.keys():
            target_switch_name = sp.slice(3,2,link_name).unwrap_some() # Assuming links
are named as `s1-s2`
```

```
            if self.data.region_switches.contains(target_switch_name):
                # Check if this target region links to the destination region
                dest_links = regions[self.data.region_switches[target_switch_name]].links
                for dest_link_name in dest_links.keys():
                    target_switch_name_1 = sp.slice(3,2,dest_link_name).unwrap_some() #
Assuming links are named as `s1-s2`
                    if self.data.region_switches.contains(target_switch_name_1):
                        if self.data.region_switches[target_switch_name_1] ==
params.destination_region_name:
                            # Calculate total latency
                            latency = starter_links[link_name].latency +
dest_links[dest_link_name].latency
                            # Find the optimal region with the least latency
                            if latency < min_latency:
                                min_latency = latency
                                optimal_region = self.data.region_switches[target_switch_name]

        self.data.optimal_routes[starter_destination] = sp.record(optimal_region =
optimal_region)
        # # send tez
        controller_address =
self.data.regions[optimal_region].controller[self.data.region_controllers[optimal_region]].AD
DRESS
        sp.send(controller_address, sp.mutez(100000))
```

The new contract has one more entrypoint to find the optimal region, each time the entrypoint is called, the caller has to pay a small fee for the optimal region used as a hop to reach destination region. Here is a detailed explanation of how this function works:

Parameters

- **starter_region_name**: The name of the region where the connection starts.
- **destination_region_name**: The name of the region where the connection is intended to go.

Verification

- **sp.verify(sp.amount == sp.tez(0.01), "Incorrect fee amount")**: Ensures that the transaction includes a fee of 0.01 tez.
- **sp.verify(self.data.regions.contains(params.starter_region_name), "No such starter region")**: Verifies that the starter region exists in the stored network information.
- **sp.verify(self.data.regions.contains(params.destination_region_name), "No such destination region")**: Verifies that the destination region exists in the stored network information.

Initial Setup

- **regions**: A local variable that holds all the region information.

- **starter_links**: Extracts the links from the starter region.
- **destination_links**: Extracts the links from the destination region.
- **optimal_region**: A variable to keep track of the optimal region found during the search.
- **min_latency**: A variable initialized to a high value (99999) to keep track of the minimum latency found.
- **starter_destination**: A key to store the result, combining the starter and destination region names.

Finding the Optimal Region

1. **Iterate over the links in the starter region**:
    - **for link_name in starter_links.keys():**: Loop through all links in the starter region.
2. **Determine the target region from the link name**:
    - **option_target_region_name = sp.slice(3,2,link_name).unwrap_some()**: Extract the target region name from the link name (assuming the link name format is h11-s1).
3. **Check if the target region is valid**:
    - **if self.data.region_switches.contains(option_target_region_name):**: Ensure that the target region is a known region.
4. **Check links in the target region**:
    - **dest_links = regions[self.data.region_switches[option_target_region_name]].links**: Get the links from the target region.
5. **Iterate over the destination links**:
    - **for dest_link_name in dest_links.keys():**: Loop through all links in the target region.
6. **Determine the switch name from the destination link**:
    - **target_switch_name_1 = sp.slice(3,2,dest_link_name).unwrap_some()**: Extract the switch name from the destination link.
7. **Check if the destination region is reached**:
    - **if self.data.region_switches.contains(target_switch_name_1):**: Ensure that the switch name is valid.
    - **if self.data.region_switches[target_switch_name_1] == params.destination_region_name:**: Check if this switch connects to the destination region.
8. **Calculate and compare latency**:

- **latency = starter_links[link_name].latency + dest_links[dest_link_name].latency**: Calculate the total latency for this path.
- **if latency < min_latency:**: If this latency is less than the previously found minimum latency, update the optimal region and minimum latency.
- **min_latency = latency**: Update the minimum latency.
- **optimal_region = self.data.region_switches[option_target_region_name]**: Update the optimal region.

9. Storing the Optimal Route

- **self.data.optimal_routes[starter_destination] = sp.record(optimal_region=optimal_region)**: Store the optimal region for the given starter-destination combination.

10. Send fee to optimal region

- **sp.send(controller_address, sp.mutez(100000))**

2. Running

To run the use case, first on one terminal, we run the server:

```
~/mininet/sdn$ python3.8 pytezos/serverUseCase.py
```

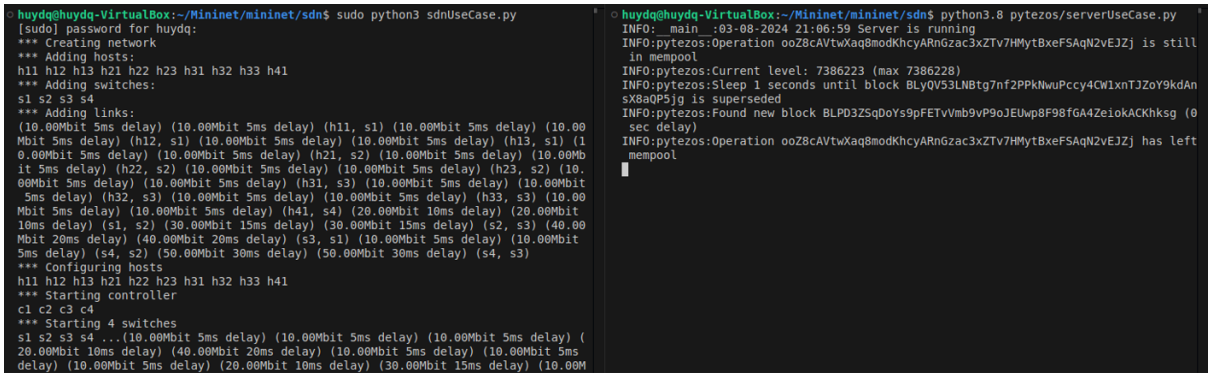
The server will start and listen to the updating network information signal:

After that, on other terminal, we run the SDN environment with updating network information thread.

```
~/mininet/sdn$ sudo python3 sdnUseCase.py
```

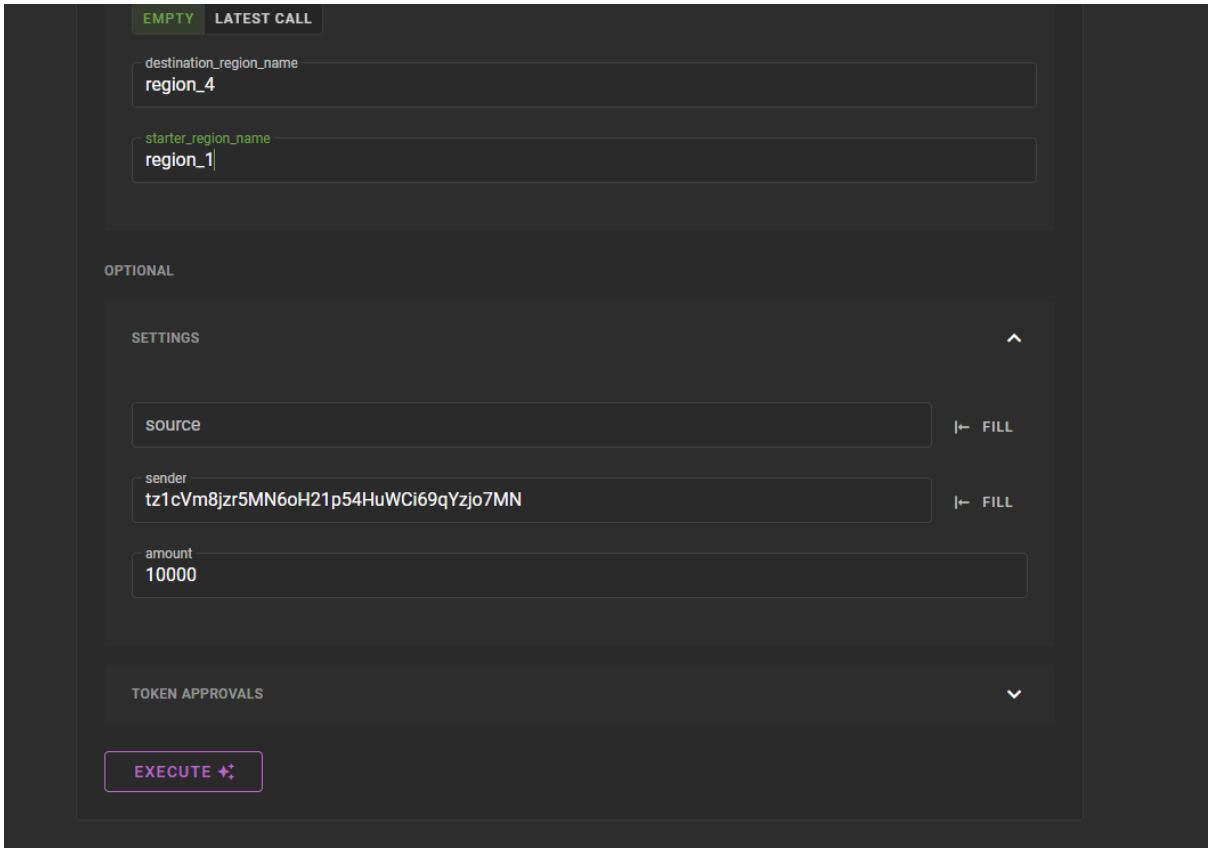
```
huydq@huydq-VirtualBox:~/Mininet/mininet/sdn$ sudo python3 sdnUseCase.py
[sudo] password for huydq:
*** Creating network
*** Adding hosts:
h11 h12 h13 h21 h22 h23 h31 h32 h33 h41
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(10.00Mbit 5ms delay) (10.00Mbit 5ms delay) (h11, s1) (10.00Mbit 5ms delay) (10.00
Mbit 5ms delay) (h12, s1) (10.00Mbit 5ms delay) (10.00Mbit 5ms delay) (h13, s1) (1
0.00Mbit 5ms delay) (10.00Mbit 5ms delay) (h21, s2) (10.00Mbit 5ms delay) (10.00Mb
it 5ms delay) (h22, s2) (10.00Mbit 5ms delay) (10.00Mbit 5ms delay) (h23, s2) (10.
00Mbit 5ms delay) (10.00Mbit 5ms delay) (h31, s3) (10.00Mbit 5ms delay) (10.00Mbit
 5ms delay) (h32, s3) (10.00Mbit 5ms delay) (10.00Mbit 5ms delay) (h33, s3) (10.00
Mbit 5ms delay) (10.00Mbit 5ms delay) (h41, s4) (20.00Mbit 10ms delay) (20.00Mbit
10ms delay) (s1, s2) (30.00Mbit 15ms delay) (30.00Mbit 15ms delay) (s2, s3) (40.00
Mbit 20ms delay) (40.00Mbit 20ms delay) (s3, s1) (10.00Mbit 5ms delay) (10.00Mbit
5ms delay) (s4, s2) (50.00Mbit 30ms delay) (50.00Mbit 30ms delay) (s4, s3)
*** Configuring hosts
h11 h12 h13 h21 h22 h23 h31 h32 h33 h41
*** Starting controller
c1 c2 c3 c4
*** Starting 4 switches
s1 s2 s3 s4 ...(10.00Mbit 5ms delay) (10.00Mbit 5ms delay) (10.00Mbit 5ms delay) (
20.00Mbit 10ms delay) (40.00Mbit 20ms delay) (10.00Mbit 5ms delay) (10.00Mbit 5ms
delay) (10.00Mbit 5ms delay) (20.00Mbit 10ms delay) (30.00Mbit 15ms delay) (10.00M
```

And finally, we have a running sdn environment with a server to push latest network information to the smart contract.

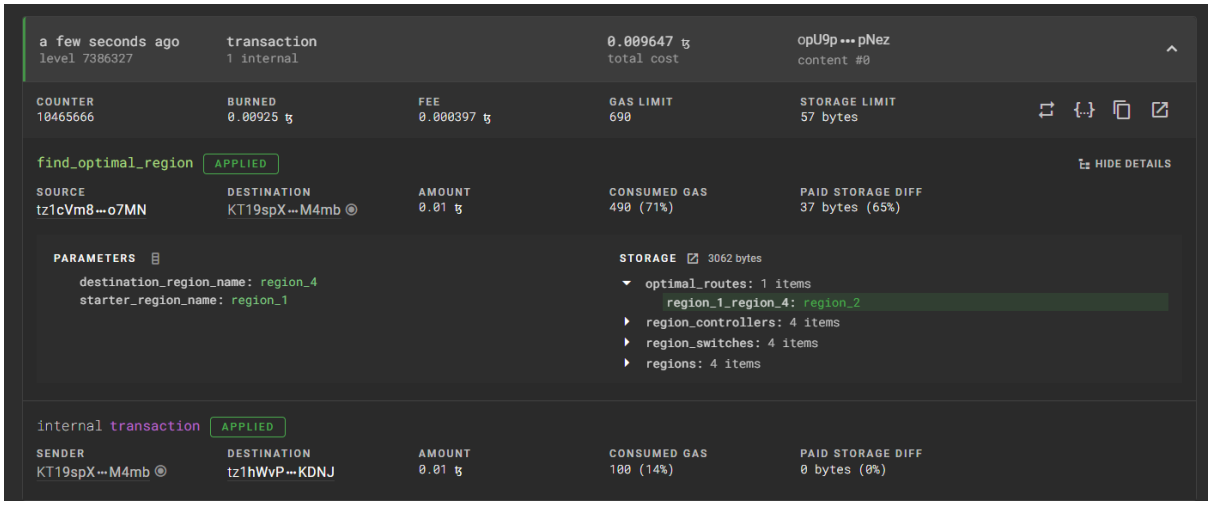


You can track the operation and current storage of the smart contract on better-call-dev.

Then we do a test to check if region 2 or region 3 is the optimal region to act as a hop to link between region 1 and region 4. On the Interact tab of better-call-dev, we input starter region name as region 1, destination region name as region 4, fill sender and fee amount as 10.000 mutez or 0.01 tez like following:



Then click EXECUTE -> Wallet and sign the transaction. After that we get following transaction hash and we can see that the optimal region is **region_2**:



The region_2 acts as a hop to link 2 region 1 and region 4 will be rewarded. And the information is also stored on the smart contract so other nodes from region 1 and region 4 can query and use region_2 as a hop to link to each region.